

MIDTERM

Airline Reservation System

Big Data and Data Engineering

1. Purpose

Before choosing AWS services or specific technologies, each team must define what the system must do (functional requirements), under what conditions it must do it (non-functional requirements), how the data is organized (entity-relationship model), and how the solution is structured (backend and frontend architecture). The exercise is designed for each team to make its own decisions and justify them. There is no single correct answer; what is evaluated is the coherence between the decisions made and the requirements the team itself defines.

2. Business scenario

A regional airline hires the team to build its flight reservation system from scratch. The business guidelines are deliberately incomplete:

- Passengers must be able to search for flights, book and pay for tickets, and view or cancel their reservations.
- The airline sells tickets directly on its website, but also through partner travel agencies.
- There are different fares depending on how far in advance the ticket is purchased and the type of seat (economy, business).
- A single flight may have layovers, and a passenger may book a multi-leg itinerary in a single transaction.
- Commercial management wants to be able to later analyze occupancy by route, revenue by fare, and cancellation patterns.
- It is mentioned that “the system must not sell more seats than the aircraft has” and that “traffic spikes during peak season,” without giving any concrete numbers.

Several details are deliberately left unspecified (number of routes, whether there is a frequent-flyer program, whether itinerary changes are allowed, what happens with baggage). Part of the exercise is deciding which assumptions to work with and documenting them explicitly: an undeclared assumption is a hidden architecture decision.

3. Team-specific constraint

Each team will receive, independently and confidentially from the other teams, an additional constraint on the general scenario described in section 2 (for example, a business condition, a technical limitation, or a specific business rule). This constraint must be coherently integrated into all of the exercise's deliverables: requirements, data model, and architecture. Consistency with this particular constraint is part of the evaluation and of the oral defense.

4. Use of artificial intelligence assistants

The use of AI assistants (Amazon Q Developer, Claude, or others) is allowed and is consistent with the course methodology, provided the student can explain and support any decision generated with their help. One additional condition is required:

- Prompt log: the final document must include, as an appendix, a brief log of the main prompts used during the exercise and how they were refined or corrected by the team.

5. Activities and decisions to make

5.1 Functional Requirements (FR)

Write the functional requirements in table format (ID, description, actor involved, priority). Guiding questions the team must resolve on its own:

- Who are the system's actors (passenger, partner agency agent, airport staff, administrator)? Do they all interact with the same system, or with different interfaces over the same backend?
- How is a reservation with multiple legs or connections handled? Is it a single reservation with several associated flights, or several chained reservations?
- Is it allowed to book without paying immediately (temporary hold on the seat), or is payment an atomic part of the reservation?
- What happens if the passenger wants to cancel or change the date?
- Do partner travel agencies have a different flow (discount, commission), or do they use the same process as an end passenger?

5.2 Non-Functional Requirements (NFR)

Define at least one non-functional requirement per category, with a verifiable metric or condition. It is not enough to write “must be fast” or “must be secure.”

Category	Question the team must resolve
Performance	What response time is acceptable for a flight availability search? Does it change if layovers are involved?
Consistency / Concurrency	If two passengers try to book the last available seat at the same time, what must the system guarantee?
Availability	What availability number (99%, 99.9%, 99.99%) translates “can't go down during peak season,” and what does that imply for cost and complexity?
Security	How are payment data and passengers' personal data protected? Does any relevant regulation apply?
Scalability	If the airline adds 20 new routes in a year, which part of the architecture would be the first bottleneck?
Auditability	How important is it to reconstruct the complete history of changes to a reservation?

5.3 Entity-relationship model

Design the entity-relationship model considering, at minimum, entities such as Flight, Aircraft, Route, Airport, Passenger, Reservation, Fare, Seat, and Payment, without assuming this list is complete or that these are exactly the right entities. Decisions the team must make and justify:

- How is a multi-leg itinerary modeled? Does a reservation have many associated flights, or is the itinerary its own entity between Reservation and Flight?
- Is seat assignment part of the reservation from the moment of purchase, or is it a separate process (check-in) that happens later?
- How is it represented that the same flight repeats every day? Is there a “scheduled flight” entity distinct from a “flight instance on a specific date”?
- Where does the control of available seat inventory live in order to prevent overselling?
- What level of normalization is justified given that this model will feed both the transactional system and the course's Big Data pipeline?

Deliverable: entity-relationship diagram using standard notation (Chen or Crow's Foot, team's choice, but consistent throughout the diagram) and a brief data dictionary per entity.

5.4 Backend architecture

- Monolith or separate services (flight search, reservation management, payments)? Which NFR supports that choice, particularly the one on concurrency over seat inventory?
- How is it guaranteed that a flight is not oversold when there are multiple simultaneous requests? Pessimistic locking, optimistic locking with retries, or a queue that serializes requests per flight?
- How is the payment flow integrated? Is it fully delegated to an external provider, or does the system handle part of the logic?
- How are services exposed to partner travel agencies? The same API as the end-passenger website, or a separate B2B API?
- How does this transactional architecture relate to the course's Big Data pipeline?

5.5 Frontend architecture

- Single-page application (SPA) or traditional server-side rendering? Which performance NFR weighs on this decision?
- How is the user notified, in real time, if the seat they were selecting is no longer available?
- Is a separate interface needed for travel agency agents?
- What implication does handling multiple currencies or languages have on the frontend structure?

5.6 First implementation of the backend

This section requires a first functional, though partial, implementation of the backend designed in 5.4: an API that exposes at least the system's critical operations, backed by a database that implements the entity-relationship model from 5.3. The purpose is not to build the entire system, but to validate that the design decisions made so far are actually implementable and consistent with each other.

Minimum required scope:

- A functional database (the engine and level of normalization are those defined and justified by the team in 5.3).

- A FastAPI API that exposes, at minimum, the operations of: searching available flights, creating a reservation, and querying an existing reservation.
- The reservation-creation operation must reflect the concurrency decision made in 5.4 (pessimistic locking, optimistic locking, queue, or another mechanism): it is not enough for the API to work in the case of a single user at a time.

Decisions the team must make and justify at this stage:

- Does the API code exactly reflect the decisions documented in 5.4, or did adjustments appear during implementation? If so, what motivated them, and which document is updated as a result?
- How was the concurrency scenario described in the 5.2 NFR (two simultaneous requests for the last seat) specifically tested? What evidence (automated test, script, execution screenshots) supports that the chosen mechanism actually prevents overselling?
- Which part of the 5.3 entity-relationship model had to be adjusted, if any, when facing the real implementation? An adjustment is not an error: it is valuable information that must be documented.

Deliverable for this subsection: API source code on GitHub and the database schema (or repository access), a brief execution guide, and concrete evidence of the concurrency test performed. Any deviation from the 5.3/5.4 design must be explicitly declared, with its justification, rather than left unmentioned.

5.7 AWS implementation: ETL, data catalog, and costs

This section requires a first real implementation that connects the transactional system to an analytical database, documenting the infrastructure decisions with the same level of justification required in previous sections.

Implementation requirements:

- The transactional database that supports the entity-relationship model designed in 5.3.
- A second PostgreSQL database, for analytical purposes (OLAP), that receives a copy of at least one table or derived view from the transactional database via an ETL process.
- Both databases must be registered in the AWS Glue Data Catalog, so that their tables are visible and queryable as metadata from Glue.

Decisions the team must make and justify:

- Which table or combination of tables makes sense to copy to the analytical database in order to answer the scenario's business questions (occupancy by route, revenue by fare, cancellation patterns)? Is it copied as-is, or transformed/aggregated along the way?
- Should the analytical database schema be identical to the transactional one, or is a different model appropriate given its analytical purpose? How does this relate to the normalization discussion in 5.3?
- Which AWS service is used to implement the ETL process? What options exist within the Learner Lab, and why choose one over another?
- How are both PostgreSQL databases connected to the Glue Data Catalog (JDBC connection, crawler, IAM role, network access)?
- Should the ETL run only once, periodically, or be event-triggered? What does each option imply for data freshness versus cost?

Justification of chosen AWS services: for each service included in this implementation (database instances, ETL service, data catalog, IAM roles, network components), the team must justify the choice by connecting it to (1) a specific functional or non-functional requirement from sections 5.1/5.2, and (2) one or more of the six pillars of the AWS Well-Architected Framework.

AWS Service	Alternative considered	Requirement supporting the choice	Well-Architected pillar(s)
(to be defined by the team)	(to be defined by the team)	(reference to an FR/NFR)	(one or more of the 6 pillars, with brief explanation)

Cost projection: prepare a monthly cost projection using the AWS pricing calculator or current public prices, including:

- Estimated cost of each database instance (transactional and analytical), with the assumed instance size/class stated.
- Estimated cost of the ETL process, based on the chosen service and the execution frequency decided.
- Estimated cost of the crawlers and the Glue catalog.
- An additional projection of how that cost would change if data volume or update frequency were multiplied by 10, explicitly connected to the cost optimization and sustainability pillars.

Deliverable for this subsection: updated architecture diagram showing both databases, the ETL process, and its relationship to the Glue Data Catalog; complete service justification table; cost projection table with explicitly stated assumptions.

6. Deliverables

Deliverable	Content
Partial submission	Functional and non-functional requirements (5.1 and 5.2)
Final submission	Complete document: context and assumptions, FR, NFR, entity-relationship model, backend and frontend architecture, first backend implementation (5.6), AWS implementation (5.7), and prompt log appendix

7. Oral defense

All team members must be able to support any part of the submitted document, including the specific constraint assigned in section 3. During the defense, the instructor may pose scenario variations not covered in the document (for example, a change in a business policy) and ask the team to explain, on the spot, which part of their design would be affected.

The oral defense is eliminatory for the corresponding section: if the team cannot coherently support a documented decision, the grade for that specific section is voided, regardless of the quality of the written document.

8. Evaluation rubric (20 points)

Criterion	Insufficient (0-40%)	Acceptable (41-70%)	Outstanding (71-100%)	Weight
Functional requirements	Generic, do not distinguish reservation from purchase nor handle multi-leg itineraries	Cover the scenario but with unresolved ambiguities	Explicitly resolve the guiding questions, state assumptions, and integrate the team-specific constraint	10%
Non-functional requirements	No verifiable metrics	Metrics present but not connected to a real business risk	Concrete metrics traceable to an explicit business risk, particularly concurrency	10%
Entity-relationship model	Does not resolve layovers or overselling control	Functional model but with unjustified decisions	Coherent model that explicitly resolves layovers, recurring flights, and seat inventory, with justification	15%
Backend and frontend architecture (design)	Technology choices without justification	Superficial justification, poorly connected to NFRs	Each decision explicitly connects to a specific NFR or FR, especially concurrency handling	15%
First backend implementation	The API or database does not work, or does not reflect the 5.3/5.4 design	Functional implementation but without evidence of the concurrency test, or without declaring deviations from the design	Functional implementation, faithful to the design (or with justified deviations), with concrete evidence of the concurrency test	10%
ETL implementation, Glue catalog, and costs	The ETL is not functional, or the databases do not appear registered in the Glue Data Catalog	Functional implementation but without clear justification of services or costs	Functional implementation, services justified with FR/NFR and a Well-Architected pillar, costs with stated assumptions and a growth scenario	20%
Clarity and traceability of the document	Disconnected sections	Complete document but without cross-references	An architecture decision can be traced back to the specific requirement that originates it	10%